D.Y. PATIL COLLEGE OF ENGINEERING & TECHNOLOGY,
KASABA BAWADA, KOLHAPUR
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

(Academic Year: 2025-26)



Report

of

Title : SmartSched – Intelligent Timetable Generator

Submitted By:

Name	Roll No.
Sarvesh Sagar Mahadik	42
Pruthviraj Jadhav	49
Anurag Kokate	43

Under the Guidance of:
Mrs. Sharayu Nilesh Patil

Class: TY **Div.: A** **Batch: T3**

CERTIFICATE

This is to certify that following students

Name	Roll
Sarvesh Sagar Mahadik	42
Pruthviraj Jadhav	49
Anurag Kokate	43

have successfully completed the term work for entitled

"SmartSched – Intelligent Timetable Generator", towards the partial fulfillment of T.Y. (CSE) Sem VI during the academic year 2025-26.

Guide	HOD	Principal
Mrs. Sharayu Nilesh Patil	Prof. R.J. Dhanal	Dr. Prof. S.D. Chede

ACKNOWLEDGEMENT

We express our sincere thanks to Mrs. Sharayu Nilesh Patil, Computer Science and Engineering Department, whose supervision, inspiration, and valuable guidance helped us a great deal in completing our Scripting Concepts Mini Project. Her guidance proved to be most valuable in overcoming all hurdles in the fulfillment of this Project Report.

We also express our sincere thanks to Prof. R.J. Dhanal, Head of the Computer Science Department, and Dr. Prof. S.D. Chede, Principal of D.Y. Patil College of Engineering & Technology, Kolhapur. Last but not least, this acknowledgement would be incomplete without rendering our sincere gratitude to all those who have helped us in the completion of this Scripting Concepts Mini Project. Finally, we express our deep sense of gratitude to our parents and family members, who are the chief source of our inspiration.

Sincerely,

Sarvesh Sagar Mahadik

Pruthviraj Jadhav

Anurag Kokate

TY CSE

Scripting Concepts Mini Project

INDEX

Chapter-1: INTRODUCTION	
1.1	Introduction
Chapter-2: PROBLEM STATEMENT	
2.1	Problem Statement
2.2	Objectives
Chapter-3: DESIGN DETAILS	
3.1	System Architecture
3.2	Database Schema
3.3	CSP Scheduling Algorithm
Chapter-4: IMPLEMENTATION	
4.1	Technology Stack
4.2	Key Modules & Code Snippets
Chapter-5: CONCLUSION	
5.1	Conclusion
5.2	Future Scope
Chapter-6: REFERENCES	
6.1	References

Chapter 1: Introduction

1.1 Introduction to SmartSched

In modern academic institutions, scheduling classes, subjects, faculty, and rooms manually is an error-prone and time-consuming process. Conflicts in timetables — such as double-booking of rooms or faculty assigned to multiple classes at the same time — lead to disruption in academic schedules and inefficient use of institutional resources.

SmartSched is a full-stack web application designed to automate the generation of conflict-free academic timetables using a Constraint Satisfaction Problem (CSP) based scheduling engine. The system allows administrators to define subjects, faculty, rooms, and divisions, and then automatically produces a valid timetable that satisfies all specified constraints.

A key feature of the platform is support for elective synchronization, enabling Professional Electives (PE) and Open Electives (OE) to be scheduled consistently across multiple divisions. The system further provides a drag-and-drop interface for manual adjustments, with real-time conflict validation ensuring the schedule remains valid at all times.

1.2 Overview of the CSP-Based Scheduling Approach

Constraint Satisfaction Problems (CSP) are a class of computational problems where a solution must satisfy a set of constraints simultaneously. Academic timetable generation is a well-known NP-hard problem that maps naturally to the CSP formulation.

In SmartSched, the CSP solver employs the Minimum Remaining Values (MRV) heuristic combined with forward checking to efficiently explore the search space. Key constraints enforced include:

- No faculty member may be assigned to more than one class at the same time slot.
- No room may be assigned to more than one class simultaneously.
- Professional and Open Electives must be synchronized across divisions to avoid resource conflicts.
- Subject credit hours must be respected across the weekly schedule.

The AI-powered data extraction module leverages the Google Gemini API to parse faculty, room, and subject information from uploaded images, significantly reducing manual data entry.

Chapter 2: Problem Statement

2.1 Problem Statement

Traditional timetable generation in academic institutions faces several significant challenges:

- Manual Scheduling Overhead: Academic coordinators spend considerable time manually assigning subjects, faculty, and rooms to time slots, with limited tooling to detect conflicts.
- Conflict Management: Without automated validation, timetables frequently contain hidden conflicts such as faculty double-booking or room unavailability, which are discovered only after deployment.
- Elective Synchronization: Scheduling electives across multiple divisions is particularly complex and is often handled through error-prone manual coordination.
- Scalability: As the number of divisions, subjects, and faculty increases, the complexity of the scheduling problem grows exponentially, making manual approaches infeasible.
- Data Entry Burden: Administrators must enter large amounts of structured data (faculty load, subject credits, room capacities) before scheduling can begin.

SmartSched addresses these challenges by providing a centralized, automated timetable generation platform that enforces constraints programmatically and provides intuitive tools for administrators to configure, generate, review, and export academic schedules.

2.2 Objectives

- To design and implement a CSP-based scheduling algorithm with MRV heuristic and forward checking for generating conflict-free timetables.
- To support multi-class timetable generation for multiple divisions simultaneously within a single scheduling run.
- To implement elective synchronization for Professional Electives (PE) and Open Electives (OE) across divisions.
- To develop a drag-and-drop interface enabling manual timetable editing with real-time conflict validation.
- To integrate the Google Gemini API for AI-powered extraction of faculty, room, and subject data from images.
- To provide export functionality supporting both Excel (.xlsx) and PDF formats for generated timetables.
- To implement role-based access control with Admin and Faculty user roles using JWT-based authentication.
- To build a responsive and intuitive frontend using React 19, TailwindCSS, and Framer Motion.

Chapter 3: Design Details

3.1 System Architecture

SmartSched follows a three-tier architecture comprising the Presentation Layer, the Business Logic Layer, and the Data Layer, consistent with modern full-stack web application design principles.

Layer	Component & Responsibility
Presentation Layer	React 19 + Vite frontend; renders timetable grids, configuration forms, and drag-and-drop editing interface.
Business Logic Layer	Node.js / Express 5 backend; runs the CSP scheduling engine, handles API routes, session management, and data validation.
Data Layer	MongoDB / Mongoose; stores subjects, faculty profiles, room definitions, divisions, and generated timetable records.
AI Integration	Google Gemini API; extracts structured data from uploaded images of existing timetables or handwritten schedules.

3.2 Database Schema

The following collections form the core data model of SmartSched:

Subjects Collection

Field	Type	Description
subject_id	ObjectId	Primary key, unique identifier.
name	String	Full name of the subject.
code	String	Short subject code (e.g., CS401).
credits	Integer	Number of credit hours per week.
type	String	Theory, Practical, PE, or OE.
faculty_id	ObjectId (FK)	Reference to the assigned faculty.

Timetable Records Collection

Field	Type	Description
timetable_id	ObjectId	Primary key for each generated timetable.
division_id	ObjectId (FK)	Reference to the division.
slots	Array	Array of slot objects (day, period, subject, faculty, room).
generated_at	DateTime	Timestamp of generation.
status	String	Draft, Published, or Archived.

3.3 CSP Scheduling Algorithm Flow

The following steps describe the end-to-end CSP-based scheduling process in SmartSched:

- Step 1 – Input Configuration: The administrator provides subjects, faculty availability, room capacities, and division requirements through the frontend forms or via AI-extracted data.
- Step 2 – Variable Assignment: Each time slot (day + period) across all divisions is treated as a CSP variable. The domain of each variable is the set of valid (subject, faculty, room) triplets.
- Step 3 – MRV Heuristic: Slots with the fewest remaining valid assignments are prioritized first, reducing backtracking.
- Step 4 – Forward Checking: After each assignment, the algorithm eliminates inconsistent values from the domains of unassigned slots.
- Step 5 – Elective Sync: PE and OE slots across divisions are assigned simultaneously to ensure no faculty or room conflict arises.
- Step 6 – Backtracking: If no valid assignment exists for a slot, the algorithm backtracks and tries alternative assignments.
- Step 7 – Output: A fully validated, conflict-free timetable is stored in the database and presented to the administrator for review.

Chapter 4: Implementation

4.1 Technology Stack

Category	Technology	Purpose
Frontend	React 19, Vite, TailwindCSS, Framer Motion	User interface, timetable display, and animations.
Drag & Drop	dnd-kit	Interactive slot rearrangement with conflict validation.
Backend	Node.js, Express 5	REST API server and CSP algorithm execution.
Database	MongoDB, Mongoose	Persistent storage for all scheduling data.
AI Extraction	Google Gemini API	Parsing faculty/room/subject data from images.
Authentication	JWT, bcryptjs	Secure role-based login for Admin and Faculty.
Export	xlsx, pdf libraries	Download timetables as Excel or PDF files.
Version Control	Git, GitHub	Source code management and collaboration.

4.2 Key Modules & Code Snippets

Module 1 – CSP Scheduler Core

The CSP scheduler applies MRV heuristic and forward checking to assign subject-faculty-room triplets to time slots without conflicts:

```
function cspSchedule(divisions, subjects, faculty, rooms) {
   const slots = initializeSlots(divisions);
   const sorted = applyMRV(slots);
   return backtrack(sorted, {}, faculty, rooms);
 }
```

Module 2 – Drag-and-Drop Editor

The drag-and-drop editor uses dnd-kit to allow administrators to move timetable slots. On each drop event, the conflict checker validates the proposed change before committing it:

```
function onDragEnd({ active, over }) {
   if (!over) return;
   const conflict = checkConflict(active.id, over.id, timetable);
   if (!conflict) dispatch(moveSlot(active.id, over.id));
   else showConflictAlert(conflict);
 }
```

Module 3 – API Route Summary

Method	Route	Description
POST	`/api/timetable/generate`	Trigger CSP scheduler to generate a new timetable.
GET	`/api/timetable/:id`	Retrieve a generated timetable by ID.
PUT	`/api/timetable/:id/slot`	Update a single slot (drag-and-drop commit).
POST	`/api/ai/extract`	Extract scheduling data from an uploaded image via Gemini.
GET	`/api/timetable/:id/export`	Export timetable as Excel or PDF.
DELETE	`/api/timetable/:id`	Delete a timetable record (Admin only).

Chapter 5: Conclusion

5.1 Conclusion

SmartSched successfully demonstrates the design and implementation of an intelligent, full-stack timetable generation system that addresses the real-world challenges of academic scheduling. By applying a Constraint Satisfaction Problem (CSP) solver with MRV heuristic and forward checking, the system generates conflict-free timetables efficiently, even for complex scenarios involving multiple divisions and elective synchronization.

The integration of the Google Gemini API for AI-powered data extraction significantly reduces the administrative burden of manually entering faculty, subject, and room information. The drag-and-drop editing interface with real-time conflict validation ensures that administrators retain full control over the final schedule while being protected from invalid configurations.

The project effectively covers the full software development lifecycle — from requirements gathering and system design to implementation and testing. The modular three-tier architecture ensures separation of concerns and makes the system maintainable and extensible.

In summary, SmartSched provides:

- An intelligent, CSP-based scheduling engine that generates conflict-free academic timetables automatically.
- AI-powered data extraction using Google Gemini, reducing manual data entry effort.
- An interactive drag-and-drop editor with real-time conflict validation.
- Multi-class and elective synchronization support for complex institutional requirements.
- Flexible export to Excel and PDF for distribution and printing.

5.2 Future Scope

While the current implementation meets the core requirements, several enhancements are planned for future iterations:

- Faculty Preference Management: Allow faculty to specify preferred teaching time slots and incorporate these as soft constraints in the CSP solver.
- Multi-Semester Planning: Extend scheduling support across semesters, enabling year-level academic planning.
- Automated Conflict Notifications: Send real-time email or push notifications to faculty when their timetable is updated or conflicts are detected.
- Advanced Analytics Dashboard: Provide administrators with visualizations of room utilization rates, faculty workload distribution, and scheduling efficiency metrics.
- Mobile Application: Develop a React Native or Flutter companion app allowing faculty and students to view their timetables on mobile devices.
- Integration with ERP Systems: Connect with institutional ERP platforms for automatic synchronization of faculty and student records.
- Containerization and Cloud Deployment: Package the application using Docker and Kubernetes for scalable deployment on cloud platforms such as AWS or Azure.

- Multi-Institution Support: Extend the platform to serve multiple institutions from a single deployment with tenant-level isolation.

Chapter 6: References

1. Russell, S. & Norvig, P. – Artificial Intelligence: A Modern Approach, 4th Ed. Pearson. (CSP Algorithm Reference)
2. React Official Documentation. Available at: https://react.dev/
3. Node.js Official Documentation. Available at: https://nodejs.org/en/docs/
4. Express.js Official Documentation. Available at: https://expressjs.com/
5. MongoDB Documentation. Available at: https://www.mongodb.com/docs/
6. Google Gemini API Documentation. Available at: https://ai.google.dev/docs
7. dnd-kit Documentation. Available at: https://dndkit.com/
8. SmartSched GitHub Repository. Available at: https://github.com/Atharv-Pawar01/ScheduleForge
9. TailwindCSS Documentation. Available at: https://tailwindcss.com/docs/